CPU Cache and the Impact to Data Structures

Harrison T Farrell

Software Engineer, Sydney, Australia.

18-Feb-2026

Abstract

Computer science data structures are usually categorized into one of two groups; Contiguous and non-contiguous structures. This articles mains to explain how the performance of these groups may, at times, be impacted by the CPU cache. The algorithms used on the structures may share the same Big O Notation in terms of time complexity. However, due to the CPU architecture, in particular, the CPU cache. Performance may differ. This article compares four common data structures from the C++ Standard Library. Demonstrating how the CPU caches effects performance.

Contents

List of Tables

Listings

1 Introduction

A CPU cache is a small, ultra-fast memory component built directly into a CPU. The purpose of a memory cache is to maximize processing speed by reducing the time it takes the CPU to search for data from the main memory.

A CPU cache is often separated into 3 clear levels *'L1, L2, L3'*. The CPU will traverses up the levels has it attempts to find the memory address value. Initially searching the L1, continuing to L2 and L3 before reaching the main memory. Table 1 shows approximate number of clock cycles and time taken to reach the a memory address at each level.

A cache miss is an event when requested data is not found in the cache memory by the CPU. Instead it must be retrieved from slower main memory (RAM or disk), resulting in increased latency and reduced performance

Memory	Cycles	Time (ns)
L1	3	1
L2	10	3
L3	100	15
RAM	250	100

Table 1: Access Time

2 Data Structures

The C++ Standard Library provides numerous number of data structure options. Four have been selected for evaluation based on two properties. Firstly, if the structure is contiguous or not, secondly the search algorithm O notation speed. Table 2 shows the four structures chosen for evaluation.

An unsorted and sorted vector were selected to allow the comparison search algorithms on similar contiguous data structures. A set was selected to allow the comparison of similar search algorithms on contiguous verse non-contiguous structure. An unordered set was selected to add another non-contiguous structure with a unique search algorithm.

Name	Type	Search
unsorted vector	contiguous	O(n)
sorted vector	contiguous	O(Log N)
set	non-contiguous	O(Log N)
unordered set	non-contiguous	O(1)

Table 2: Library Containers

3 Search Algorithms

Unsorted Vector

When calling std::find on an unsorted std::vector. The C++ Standard Template Library (STL) will perform a linear search. Since the vector isn't sorted, the algorithm has no "shortcuts" and must check each element one by one. Under the hood std::find iterates through the range [first,last] and compares each element to the target value provided. It stops when once it finds a match or reaches the end iterator. When the CPU fetches one element, it pre-loads the neighboring elements into the cache. This means the "next" increment in the loop is usually already sitting in the high-speed cache, making the linear sweep extremely performant for small-to-medium datasets.

Listing 1: Vector Linear Search

```
// std::find as a linear search O(N)
bool linearSearch(std::vector<int>
    &data_set, int target) {
  if (std::find(data_set.begin(),
      data_set.end(), target) !=
      data_set.end()) {
    return true;
  }
  return false;
}
```

Sorted Vector

Under the hood when calling std::binary_search on a sorted std::vector it performs a binary search algorithm. Instead of starting at the beginning, std::binary_search jumps straight to the middle. It compares the middle element to the target value. If the target is smaller: It ignores the right half and repeats the process on the left half. If the target is larger: It ignores the left half and repeats on the right half. While std::binary_search performs fewer comparisons, it is less "cache-friendly" than std::find. Because it jumps to distant memory addresses, it often triggers cache misses.

Listing 2: Vector Binary Search

```
// std::binary_search as a red black
    tree search O(Log N)
bool binary_search(const
    std::vector<int>& data_set, int
    target) {
  return
      std::binary_search(data_set.begin(),
      data_set.end(), target);
}
```

Set, std::set::find

Since a std::set is typically implemented as a Red-Black Tree, The member function uses the inherent structure of the tree to locate the target value. Instead of checking every element from "left to right," the function starts at the Root Node and follows a specific path based value at the given node. If value is less than the current node, move to the Left Child. If value is greater than the current node, move to the Right Child. While the logic is mathematically superior to a linear search, there are physical hardware realities.

- Pointer Chasing: As each node in the tree is a separate memory allocation. Moving from parent to child requires following a pointer to a potentially distant memory address.
- Cache Misses: As these nodes are non-contiguous, the CPU often has to wait for data to be fetched from RAM, which is much slower than the CPU cache.

Listing 3: Set Binary Search

```
// std::set::find as a red black tree
    search O(Log N)
bool setSearch(const std::set<int>&
    data_set, int target)
{
  auto search = data_set.find(target);
  return search != data_set.end();
}
```

Unordered Set

When calling the member find method on an unordered set. The method passes target value into a hash function. This converts the value into a large integer. It takes that large integer and performs a modulo operation to determine which "Bucket" the value belongs in. It jumps straight to that bucket in memory. Since multiple values could hash to the same bucket (a "collision"), each bucket usually contains a small linked list. The function does a quick linear search through that specific bucket's list to find the exact match. Table 3 shows the growth rate of buckets as the element count grows.

Elements	Buckets
8	13
16	29
34	59
64	127
128	257
256	541
512	1109
1024	1109

Table 3: Bucket Growth

The C++ Standard forces std::unordered_set to implemented using Node-Based Chaining. The top-level array of buckets is contiguous in memory, a cache-friendly structure. These buckets contain nodes that are scattered across the main memory (RAM). To search for a value. The algorithm steps into a bucket and traverses a linked lists searching for the value. Every time the CPU "follows a pointer" to a new memory address, it risks a Cache Miss.

Listing 4: Unordered Set Search

```
// std::unordered_set::find as a hashmap
    search O(1)
bool unorderedSearch(const
    std::unordered_set<int>& data_set,
    int target) {
  auto search = data_set.find(target);
  return search != data_set.end();
}
```

search, tree-based, hash-table and memory-latency-dominated costs as container size grows. The experiments was performed 5 times and then averaged. Table 4 shows the results for each search

Size	Unordered Set	Sorted Vec	Set	Vec
8	22.9	13.3	40.4	8.4
16	21.3	15.2	51.7	9.8
32	22.3	17.7	61.9	25.4
64	21.0	26.6	65.7	49.7
128	22.3	27.1	68.2	78.9
256	22.0	30.0	71.1	161.4
512	21.8	34.3	73.0	333.5
1024	21.9	35.5	73.6	492.8

Table 4: Search Results (ns)

4 Benchmarks Results

A suite ran four Google Benchmark experiments to compare lookup strategies and memory-behavior. A summary of the four are listed below.

- linear scan over a shuffled std::vector for sizes 8→1024
- binary search on a sorted contiguous std::vector for sizes 8→1024
- red-black-tree binary search on an ordered std::set for sizes 8→1024
- hash-map lookup on an unordered_set for sizes 8→1024

Each benchmark builds its data set, selects a random target with std::uniform_int_distribution(0, n), then repeatedly measures a single lookup operation inside the benchmark loop using benchmark::DoNotOptimize to prevent elision; the intent is to isolate and compare linear-scan, binary-

The benchmark results reveal a stark contrast between theoretical complexity and physical hardware limits, particularly regarding how spatial locality influences latency. At small scales (N=8 to 16), the unsorted vector is the top performer, clocking in between 8.4 ns and 9.8 ns because its contiguous memory allows the CPU to preload neighboring elements into the ultra-fast L1 cache. However, as the data set grows to 1024 elements, the vector's O(N) linear scan suffers from a significant bottleneck, jumping to 492.8 ns. In contrast, the unordered set maintains remarkable stability, hovering around 21–22 ns regardless of size, though it is initially slower than vectors due to the overhead of hash functions and bucket management. The most telling comparison lies between the sorted vector and the set; despite both utilizing O(logN) logic, the sorted vector is more than twice as fast at 1024 elements (35.5 ns vs 73.6 ns). This disparity is driven by pointer chasing in the set's non-contiguous tree structure, which frequently triggers 100 ns cache misses by forcing the CPU to fetch data from RAM rather than the local cache.

5 Conclusion

The benchmark results presented in this study demonstrate that the theoretical time complexity (Big O notation) of a data structure does not always dictate its real-world performance on modern hardware. The interaction between data layout and the CPU cache hierarchy plays a pivotal role in execution speed, as evidenced by the significant latency differences between L1 cache (1 ns) and RAM (100 ns).

Contiguous structures, specifically the std::vector, benefit immensely from spatial locality. During a linear search, the CPU is able to preload neighboring elements into the cache, allowing even an O(n) algorithm to outperform more mathematically "superior" structures at smaller scales. For instance, at a size of 16 elements, the linear vector search (9.8 ns) was nearly twice as fast as the O(1) unordered set (21.3 ns).

Conversely, non-contiguous structures like std::set and std::unordered_set suffer from pointer chasing. Although a std::set uses a binary search logic similar to a sorted vector, its node-based architecture frequently triggers cache misses as the CPU follows pointers to distant memory addresses. This is reflected in the benchmarks, where the sorted vector consistently outperformed the set across all tested sizes

Ultimately, when selecting a data structure, engineers must weigh algorithmic complexity against the physical realities of the CPU cache to ensure optimal software performance.

Container	Memory Type	Search Complexity	Cache-Friendliness
Unsorted Vector	Contiguous	O(n)	High: Preloads neighbors
Sorted Vector	Contiguous	O(logN)	Medium: Jumps cause some misses
Set	Non-contiguous	O(logN)	Low: Pointer chasing
Unordered Set	Non-contiguous	O(1)	Low: Node-based chaining

Table 5: Cache Container Impact Summary